

Swinburne University of Technology*Faculty of Science, Engineering and Technology***FINAL EXAM COVER SHEET**

Subject Code: COS30008
Subject Title: Data Structures & Patterns
Due date: June 24, 2020, 20:59
Lecturer: Dr. Markus Lumpe

Your name: _____ **Your student id:** _____

Check Tutorial	Mon 14:30	Tues 08:30	Tues 10:30	Tues 12:30	Tues 14:30	Wed 08:30	Wed 10:30	Wed 12:30	Wed 14:30	Thurs 08:30

Marker's comments:

Problem	Marks	Obtained
1 – TTree	104	
2 - traversDepthFirst	14	
3 – LinearVisitor	16	
4 – Knowledge Questions	20	
Total	154	

This test requires approx. 2 hours and accounts for 50% of your overall mark.

Problem 1**(104 marks)**

We wish to define a generic 3-ary tree in C++. We shall call this data type `TTree`. A 3-ary tree has a key `fKey` and three nodes `fLeft`, `fMiddle`, and `fRight`. Following the principles underlying the definition of general trees, a 3-ary tree is a finite set of nodes and it is either

- an empty set, or
- a set that consists of a root and exactly 3 distinct 3-ary subtrees.

Somebody has already started with the implementation and created the header file `TTree.h`, but left the project unfinished. Complete the following code fragments. In particular, make sure that all elements are properly defined and no linker errors occur.

```
#pragma once

#include <stdexcept>

template<class T>
class TTree
{
private:
    T fKey;
    TTree<T>* fLeft;
    TTree<T>* fMiddle;
    TTree<T>* fRight;

    TTree() : fKey()           // use default constructor to initialize fKey
    {
        fLeft = &NIL;         // loop-back: The sub-trees of a TTree object with
        fMiddle = &NIL;        // no children point to NIL.
        fRight = &NIL;
    }

    void addSubTree( TTree<T>** aBranch, const TTree<T>& aTTree )
    {
        if ( *aBranch != &NIL )
        {
            delete *aBranch;
        }

        *aBranch = (TTree<T>*)&aTTree;
    }

    const TTree<T>& removeSubTree( TTree<T>** aBranch );           // 14 marks

public:
    static TTree<T> NIL;     // sentinel                             // 5 marks

    // TTree constructor (takes one argument)                       // 8 marks
    TTree( const T& aKey );

    // destructor (free sub-trees, must not free empty TTree)       // 15 marks
    ~TTree();

    // copy constructor, must not copy empty TTree                  // 20 marks
    TTree( const TTree<T>& aOtherTTree );

    // assignment operator, must not copy empty TTree               // 20 marks
    TTree<T>& operator=( const TTree<T>& aOtherTTree );

    // clone TTree, must not copy empty TTree                       // 12 marks
    TTree<T>* clone() const;
```

```
// return key value, may throw domain_error if empty           // 6 marks
const T& getKey() const;

// returns true if this TTree is empty                         // 4 marks
bool isEmpty() const;

// getters for subtrees
const TTree<T>& getLeft() const
{
    return *fLeft;
}

const TTree<T>& getMiddle() const
{
    return *fMiddle;
}

const TTree<T>& getRight() const
{
    return *fRight;
}

// add a subtree
void addLeft( const TTree<T>& aTTree )
{
    addSubTree( &fLeft, aTTree );
}

void addMiddle( const TTree<T>& aTTree )
{
    addSubTree( &fMiddle, aTTree );
}

void addRight( const TTree<T>& aTTree )
{
    addSubTree( &fRight, aTTree );
}

// remove a subtree, may through a domain error
const TTree<T>& removeLeft()
{
    return removeSubTree( &fLeft );
}

const TTree<T>& removeMiddle()
{
    return removeSubTree( &fMiddle );
}

const TTree<T>& removeRight()
{
    return removeSubTree( &fRight );
}
};
```

The class `TTree` is rather complex and uses explicit pointers to its three subtrees: `fLeft`, `fMiddle`, and `fRight`. To facilitate adding and removing subtrees, the class `TTree` defines two private methods, `addSubTree` and `removeSubTree`, which, in the first argument, take a pointer to the branch being altered (a pointer to a pointer of `TTree<T>`). So, the methods `addSubTree` and `removeSubTree` receive the address of the subtree member to be updated. We have seen this concept previously, while building a singly-linked list in class.

Unfortunately, only `addSubTree` has been implemented. Use the available information to implement `removeSubTree` also. The method `removeSubTree` has to guarantee that empty trees are not removed. In this case, `removeSubTree` has to throw a domain error. If

the subtree can be removed, then a constant reference to it must be returned. In addition, the pointer of the subtree being removed must be set the address of `NIL` to indicate that this branch is now empty.

Class `TTree` has to implement proper copy semantics. For this reason, it must define a destructor, a copy constructor, and an assignment operator. In addition, we require a `clone` method to easily duplicate whole `TTrees`. The copy semantics and clone must not create copies of `NIL`. If `NIL` is encountered in the copy constructor or assignment operator, then a domain error must be thrown. The method clone can easily prevent copies of `NIL` by returning the `this` object. You may need to apply suitable casts where necessary to make the implementation compile.

To create `TTree` objects, we need to define its constructor. There are two service functions: `getKey` and `isEmpty` that return the payload of the `TTree` object and test whether the current `TTree` object is the empty tree, respectively.

Finally, remember the specific C++ requirements to initialize a static member variable.

You can use the following test driver to verify your implementation:

```
string s1( "A" );
string s2( "B" );
string s3( "C" );
string s4( "D" );

TTree<string> treeA( s1 );
TTree<string>* treeB = new TTree<string>( s2 );
TTree<string>* treeC = new TTree<string>( s3 );
TTree<string>* treeD = new TTree<string>( s4 );

cout << "The payload of treeA: " << treeA.getKey() << endl;

try
{
    treeA.getMiddle().getKey();

    cout << "You should not see this output." << endl;
}
catch (domain_error e)
{
    cout << "Exception: " << e.what() << endl;
}

treeA.addLeft( *treeB );
treeA.addMiddle( *treeC );
treeA.addRight( *treeD );

cout << "Trees attached" << endl;
cout << "Use copy constructor." << endl;

try
{
    TTree<string> No = treeA.getLeft().getLeft();

    cout << "You should not see this output." << endl;
}
catch (domain_error e)
{
    cout << "Exception: " << e.what() << endl;
}

TTree<string> tree_copy = treeA;
TTree<string> B( tree_copy );
```

```
cout << "Copy constructors used." << endl;
cout << "Use assignment operator." << endl;

try
{
    B = treeA.getLeft().getLeft();

    cout << "You should not see this output." << endl;
}
catch (domain_error e)
{
    cout << "Exception: " << e.what() << endl;
}

const TTree<string>* lRight = &treeA.getRight();
B = treeA;

if ( &B.getRight() == lRight )
{
    cout << "You should not see this output." << endl;
}

cout << "Assignment operator used." << endl;

const TTree<string>* lLeft = &B.getLeft();
B = B;

if ( &B.getLeft() != lLeft )
{
    cout << "You should not see this output." << endl;
}

cout << "Self assigned used." << endl;
```

This code should produce an output similar to the following:

```
The payload of treeA: A
Exception: Empty TTree encountered.
Trees attached
Use copy constructor.
Exception: Copying NIL.
Copy constructors used.
Use assignment operator.
Exception: Copying NIL.
Assignment operator used.
Self assigned used.
```

No error should occur at the end.

Problem 2**(14 marks)**

Consider the following visitor class.

```
#pragma once

#include <iostream>

template<class T>
class TreeVisitor
{
public:
    virtual ~TreeVisitor() {}          // virtual default destructor

    // default behavior
    virtual void preVisit( const T& aKey ) const {}

    virtual void postVisit( const T& aKey ) const {}

    virtual void inVisit( const T& aKey ) const {}

    virtual void visit( const T& aKey ) const
    {
        std::cout << " " << aKey;
    }

    void emitNIL() const
    {
        std::cout << " []";
    }
};
```

Assume that the 3-ary tree `TTree` is fully functional.

We now wish to add a depth-first traversal facility to `TTree`. The corresponding method signature is

```
void traverseDepthFirst( TreeVisitor<T>& aVisitor ) const;
```

Depth-first traversal for 3-ary trees first checks that the current tree node is not empty. If this is the case, then we need to perform three steps:

- Visit the current node using pre-order mode,
- Traverse all sub-tree nodes of the current `TTree` node, and
- Visit the current node using post-order mode.

On empty tree nodes, just invoke the `aVisitor`'s `emitNIL` method.

There is no test yet. To test the implementation, proceed with Problem 3.

Problem 3:**(16 marks)**

Using the `TreeVisitor` shown in Problem 2 as super class, define a class `LinearVisitor` that overrides the methods `preVisit` and `postVisit`:

- `preVisit` has to output string " [" first and then calls visit with `aKey`
- `postVisit` has to output string "]"

The following test driver allows you to verify your implementation of both `TTree`'s `traverseDepthFirst` method and class `LinearVisitor`:

```
string s1( "A" );
string s2( "B" );
string s3( "C" );
string s4( "D" );
string s5( "E" );
string s6( "F" );

TTree<string> treeA( s1 );
TTree<string>* treeB = new TTree<string>( s2 );
TTree<string>* treeC = new TTree<string>( s3 );
TTree<string>* treeD = new TTree<string>( s4 );
TTree<string>* treeE = new TTree<string>( s5 );
TTree<string>* treeF = new TTree<string>( s6 );

treeA.addLeft( *treeB );
treeA.addMiddle( *treeC );
treeA.addRight( *treeD );
((TTree<string>&)treeA.getLeft()).addMiddle( *treeE );
((TTree<string>&)treeA.getRight()).addMiddle( *treeF );

LinearVisitor<string> lVisitor;

cout << "Visitor test:";

treeA.traverseDepthFirst( lVisitor );

cout << endl;
```

This code should produce an output similar to the following:

```
Visitor test: [ A [ B [] [ E [] [] [] ] [] ] [ C [] [] [] ] [ D [] [ F [] [] [] ] [] ] ]
```

Problem 4**(20 marks)**

Answer the following questions in one or two sentences:

- a. How can we avoid aliases to objects?

4a)

- b. What are the elements of proper copy control in C++?

4b)

- c. What is the difference between a stack and a queue?

4c)

- d. Why can we locate, on average, an element in a binary tree faster than in an array?

4d)

- e. Explain briefly the concept of abstract data type.

4e)

- f. It is safe to return references to local variables from a function, or not? Elaborate briefly.

4f)

- g. What is the purpose of the Visitor pattern?

4g)

- h. Is quick sort always faster than bubble sort? Explain briefly.

4h)

- i. What is required to test the equivalence of iterators?

4i)

- j. When do we need to implement a state machine?

4j)